

IEEE-CIS Fraud Risk Analytics

Big Data Final Project Report

Team 20

Ekaterina Petrova – Data Engineer

Nikita Tsukanov – Data Scientist

Olga Puzhalina – ML Specialist

Alina Sukhoverkova – Tester and Technical Writer

12 May 2026

Project Repository:

[GitHub Repository](#)

Dataset Source:

[IEEE-CIS Fraud Detection Dataset \(Kaggle\)](#)

Contents

1	Introduction	3
1.1	Business Objectives	3
2	Data Description	3
2.1	Data Characteristics	4
3	Architecture of Data Pipeline	5
3.1	The Input and the Output for Each Stage	6
4	Data Preparation	7
4.1	ER Diagram	7
4.2	Some Samples from the Database	8
4.3	Creating Hive Tables and Preparing the Data for Analysis	9
5	Data Analysis	10
5.1	Analysis Results	10
5.2	Charts	10
5.2.1	Fraud Rate by Product Category	10

5.2.2	Fraud Rate and Average Amount by Transaction Amount Band . . .	11
5.2.3	Fraud Rate by Card Network (<code>card4</code>) and Card Type (<code>card6</code>) . . .	12
5.2.4	Daily Transaction Volume	13
5.2.5	Email Domain Fraud Pattern	14
5.3	Interpretation	14
6	ML Modeling	15
6.1	Feature Extraction and Data Preprocessing	15
6.2	Training and Fine-tuning	16
6.3	Evaluation	17
7	Data Presentation	18
7.1	The Description of the Dashboard	18
7.2	Description of Each Chart	19
7.3	Findings	20
8	Conclusion	20
8.1	Limitations and Engineering Decisions	20
8.2	Reflections on Own Work	21
8.3	Challenges and Difficulties	21
8.4	Future work	21
8.5	The Table of Contributions of Each Team Member	22

1 Introduction

Fraud detection is the process of identifying suspicious transactions, activities, or behavioral patterns that may indicate financial fraud. In the modern digital economy, where payments, transfers, and online purchases are carried out continuously and at large scale, this problem becomes especially important. As the volume of financial transactions grows, suspicious behavior becomes harder to track manually, while the risks and financial losses for companies continue to increase. According to the U.S. Federal Trade Commission, reported consumer losses to fraud exceeded \$12.5 billion in 2024, which confirms that this is a real and growing practical problem rather than only a theoretical one.

Because of this, we became interested in the problem and decided to examine it more deeply from a Big Data perspective. Our goal was to analyze fraud-related patterns in transaction data, understand which signals may indicate higher fraud risk, and build a practical technical solution for this task.

As a result, we developed an end-to-end big data pipeline based on the IEEE-CIS Fraud Detection dataset. The pipeline starts from raw CSV files, loads them into PostgreSQL, imports the prepared tables into HDFS with Sqoop, creates analytical layers in Hive and Spark SQL, performs exploratory data analysis, trains distributed Spark ML models, and prepares a Superset dashboard for final business presentation.

1.1 Business Objectives

The main business objectives are:

- Build a reproducible big data pipeline from raw transaction data to analytical outputs.
- Identify transaction, payment, email-domain, and temporal patterns associated with higher fraud risk.
- Prepare partitioned and bucketed analytical storage for scalable EDA.
- Train and compare distributed machine learning models for binary fraud-risk classification.
- Present data insights and model results in a dashboard suitable for business stakeholders.

From a business point of view, the final outputs support risk monitoring and faster review prioritization. For example, transactions from riskier product categories, unusual amount bands, suspicious email domains, or high-risk card combinations can be flagged for closer inspection.

2 Data Description

We used the **IEEE-CIS Fraud Detection** dataset from Kaggle. The data is provided in CSV format and contains transaction-level records together with many additional attributes that may help identify fraudulent behavior.

Each row represents one transaction. The main target variable is `isFraud`, where:

- 0 means a legitimate transaction;
- 1 means a fraudulent transaction.

The dataset is split into several source files:

Table 1: Main source files used

File	Description
<code>train_transaction.csv</code>	Labeled transaction records with <code>isFraud</code> , transaction, card, address, C, M, and V features
<code>train_identity.csv</code>	Identity, browser, and device-related features linked to training transactions

The `sample_submission.csv` file was not loaded into the analytical database because it is only a competition template and does not contain raw labeled source data.

The dataset includes several feature groups:

- transaction information, such as `TransactionAmt` and time-derived attributes;
- product category information, such as `ProductCD`;
- payment card metadata, such as `card4` and `card6`;
- payer email domain features, such as `P_emaildomain`;
- identity, browser, and device fields from identity tables;
- many anonymized numerical variables, especially V-series and C-series features.

This dataset is well suited for a big data project because it contains a large number of rows, hundreds of columns, mixed data types, many missing values, and a realistic fraud-detection target.

2.1 Data Characteristics

The Stage I pipeline imported four source tables:

Table 2: Imported source tables

Table	Rows
<code>train_transaction</code>	590,540
<code>train_identity</code>	144,233

The generated PostgreSQL DDL contains 898 lines because the transaction tables have hundreds of columns. The important structural properties are:

- `TransactionID` is the primary key in all four source tables.
- `train_identity.TransactionID` is a foreign key to `train_transaction.TransactionID`.
- `train_transaction.isFraud` is the supervised target and is stored as `SMALLINT`.
- Many features have missing values, high cardinality, or anonymized names, so dynamic schema generation and automated feature filtering were required.

The dataset has several important structural properties:

- `TransactionID` is the key transaction identifier;
- `train_identity.TransactionID` is linked to `train_transaction.TransactionID`;
- `test_identity.TransactionID` is linked to `test_transaction.TransactionID`;
- `isFraud` is the supervised label used in Stage III;
- the dataset contains many missing values, sparse columns, and anonymized fields;
- fraud is strongly imbalanced, so evaluation cannot rely only on accuracy.

The main characteristics explored are:

- fraud rate by product category;
- fraud rate by transaction amount band;
- fraud rate by card network and card type;
- fraud pattern by email domain;
- daily fraud activity and daily transaction volume;
- selected and removed feature groups before model training.

3 Architecture of Data Pipeline

The full pipeline starts with raw IEEE-CIS CSV files and ends with a Superset dashboard. Figure 1 shows the overall architecture.

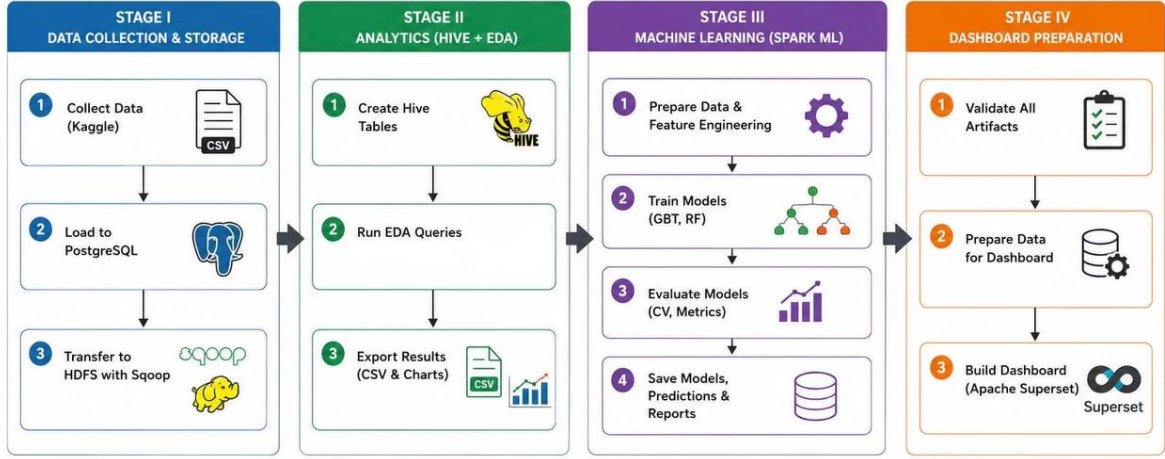


Figure 1: Architecture of the Big Data fraud detection pipeline.

The pipeline contains four main stages:

- **Stage I: Data Collection and Storage.** CSV files are validated, loaded into PostgreSQL, and imported into HDFS using Sqoop with Avro and compression support.
- **Stage II: Hive Analytics and EDA.** Hive-compatible tables are prepared, analytical tables are created, and EDA queries are executed.
- **Stage III: Machine Learning with Spark ML.** Data is joined, cleaned, transformed into feature vectors, balanced, and used to train and evaluate distributed ML models.
- **Stage IV: Dashboard Preparation.** EDA and ML artifacts are converted into datasets suitable for Apache Superset.

3.1 The Input and the Output for Each Stage

Table 3: Input and output for each project stage

Stage	Input	Output
Stage I: Collection and ingestion	IEEE-CIS CSV files in the local data folder	Generated PostgreSQL schema, loaded PostgreSQL tables, HDFS Avro warehouse, Sqoop schema files
Stage II: Storage preparation and EDA	HDFS Avro files and uploaded schema files	External Hive-compatible tables, analytical ORC table, EDA result tables, exported CSV and chart artifacts

Stage	Input	Output
Stage III: ML modeling	Hive analytical tables and prepared joined data	Trained GBT and RF models, evaluation outputs, feature importance files, prediction outputs
Stage IV: Presentation	EDA CSV/JPG outputs and ML result files	Superset-ready dashboard datasets, exported presentation tables, final dashboard plan

4 Data Preparation

Data preparation was performed before both analysis and machine learning. The goal of this stage was to transform the raw input into a clean, structured, and reusable analytical dataset.

The main preparation tasks included:

- validating CSV structure and required files;
- dynamically generating the PostgreSQL DDL from raw headers;
- loading source files into PostgreSQL;
- importing all required tables into HDFS through Sqoop;
- creating analytical views and helper tables;
- checking missing values and unknown categories;
- creating transaction amount bands;
- preparing time-based aggregates;
- filtering unusable, empty, or constant features before ML.

4.1 ER Diagram

The relational part can be represented by four main source entities:

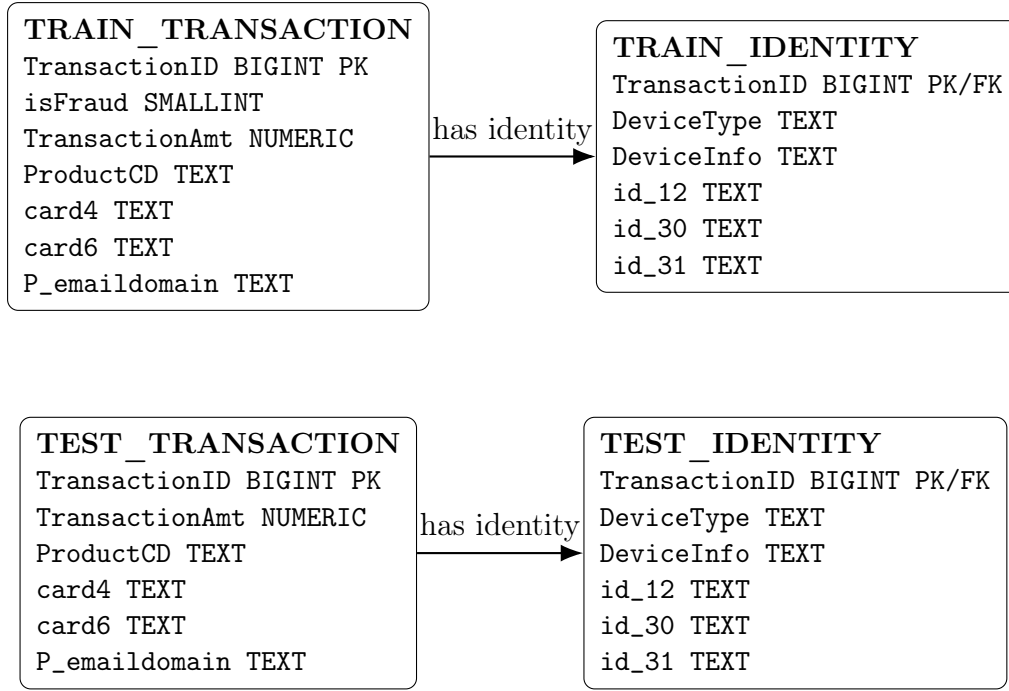


Figure 2: Representative ER diagram. The generated schema contains the full IEEE-CIS feature set.

4.2 Some Samples from the Database

After loading the data into PostgreSQL and preparing the analytical layer, sample rows were checked to verify that the pipeline produced meaningful outputs.

Example query:

```
SELECT *
FROM team20_projectdb.train_transaction_pb
LIMIT 10;
```

Example analytical result rows by product category:

Table 4: Example rows from product-category analytical output

ProductCD	Total transactions	Fraud transactions	Fraud rate	Avg amount
C	68,519	8,008	0.1169	42.87
S	11,628	686	0.0590	60.27
H	33,024	1,574	0.0477	73.17
R	37,699	1,426	0.0378	168.31
W	439,670	8,969	0.0204	153.16

Example rows by transaction amount band:

Table 5: Example rows from amount-band analytical output

Amount band	Total transactions	Fraud transactions	Fraud rate	Avg amount
00_under_25	43,329	3,019	0.0697	16.36
01_25_99	304,928	9,079	0.0298	52.19
02_100_249	175,923	5,198	0.0295	142.28
03_250_499	42,036	2,218	0.0528	338.30
04_500_plus	24,324	1,149	0.0472	981.11

4.3 Creating Hive Tables and Preparing the Data for Analysis

After the data was imported into HDFS, Hive-compatible tables were created over the stored files. This stage made it possible to run SQL-like analysis on distributed data.

The main prepared structures were:

- source external tables for the imported transaction and identity files;
- a prepared analytical table `train_transaction_pb`;
- result tables `q1_results` to `q5_results` for EDA outputs.

The prepared analytical table used:

- **partition column:** `is_fraud`;
- **bucket column:** `transactionid`;
- **storage format:** ORC for faster analytical querying.

Example table creation logic:

```
CREATE EXTERNAL TABLE fraud_transactions_hive (
  transaction_id STRING,
  transaction_day INT,
  transaction_amount DOUBLE,
  product_category STRING,
  card_network STRING,
  email_domain STRING,
  is_fraud INT
)
STORED AS ORC
LOCATION '/user/team20/fraud_transactions';
```

5 Data Analysis

The analysis focused on the following questions:

- How does fraud rate vary across product categories?
- How does fraud risk change across transaction amount bands?
- Which card network and card type combinations are associated with higher fraud risk?
- Which payer email domains show elevated fraud rates?
- How do fraud rate and transaction volume change over time?

5.1 Analysis Results

The main EDA results are the following:

- Product category C has the highest fraud rate at about 11.69%.
- Fraud risk by amount band follows a two-peak pattern: transactions under \$25 and above \$250 are riskier than the middle bands.
- Fraud risk differs across card network and card type combinations.
- Some email domains, such as `mail.com`, `outlook.es`, `aim.com`, and `outlook.com`, show elevated fraud rates.
- Fraud rate and transaction volume vary over time.

These results suggest that fraud is associated with product category, transaction amount, payment metadata, email-domain behavior, and temporal patterns.

5.2 Charts

5.2.1 Fraud Rate by Product Category

Hive query:

```
USE team20_projectdb;

DROP TABLE IF EXISTS q1_results;

CREATE TABLE q1_results
STORED AS ORC
TBLPROPERTIES ('ORC.COMPRESS'='SNAPPY')
AS
```

```

SELECT
    productcd,
    COUNT(*) AS total_transactions,
    SUM(is_fraud) AS fraud_transactions,
    SUM(is_fraud) / COUNT(*) AS fraud_rate,
    AVG(transactionamt) AS avg_amount
FROM team20_projectdb.train_transaction_pb
GROUP BY productcd
ORDER BY fraud_rate DESC;

```

```

SELECT * FROM q1_results;

```

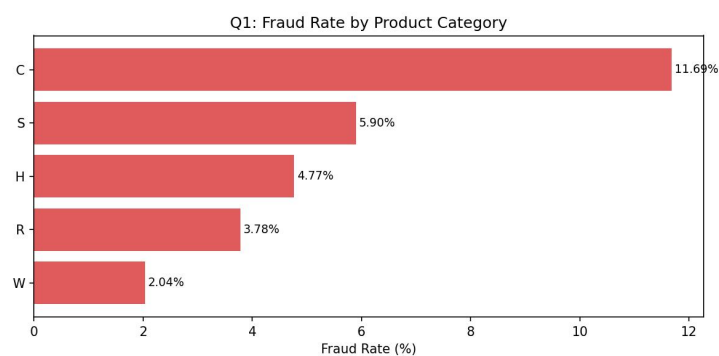


Figure 3: Fraud rate by product category.

5.2.2 Fraud Rate and Average Amount by Transaction Amount Band

Hive query used:

```

USE team20_projectdb;

DROP TABLE IF EXISTS q2_results;

CREATE TABLE q2_results
STORED AS ORC
TBLPROPERTIES ('ORC.COMPRESS'='SNAPPY')
AS
SELECT
    CASE
        WHEN transactionamt < 25 THEN '00_under_25'
        WHEN transactionamt < 100 THEN '01_25_99'
        WHEN transactionamt < 250 THEN '02_100_249'
        WHEN transactionamt < 500 THEN '03_250_499'
        ELSE '04_500_plus'
    END AS amount_band,

```

```

COUNT(*) AS total_transactions,
SUM(is_fraud) AS fraud_transactions,
SUM(is_fraud) / COUNT(*) AS fraud_rate,
AVG(transactionamt) AS avg_amount
FROM team20_projectdb.train_transaction_pb
GROUP BY
CASE
    WHEN transactionamt < 25 THEN '00_under_25'
    WHEN transactionamt < 100 THEN '01_25_99'
    WHEN transactionamt < 250 THEN '02_100_249'
    WHEN transactionamt < 500 THEN '03_250_499'
    ELSE '04_500_plus'
END
ORDER BY amount_band;

SELECT * FROM q2_results;

```

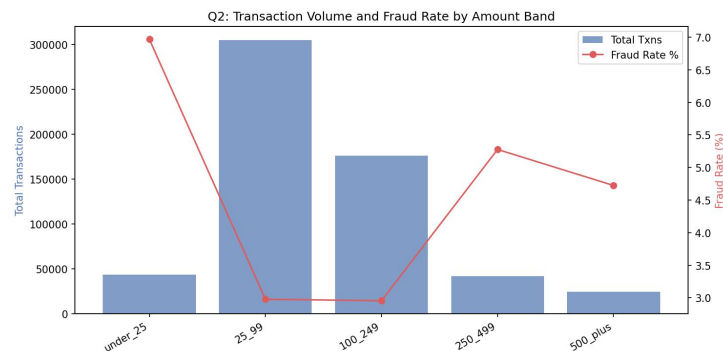


Figure 4: Fraud rate by transaction amount band.

5.2.3 Fraud Rate by Card Network (card4) and Card Type (card6)

Hive query used:

```

USE team20_projectdb;

DROP TABLE IF EXISTS q3_results;

CREATE TABLE q3_results
STORED AS ORC
TBLPROPERTIES ('ORC.COMPRESS'='SNAPPY')
AS
SELECT
    card4,
    card6,

```

```

COUNT(*) AS total_transactions,
SUM(is_fraud) AS fraud_transactions,
SUM(is_fraud) / COUNT(*) AS fraud_rate,
AVG(transactionamt) AS avg_amount
FROM team20_projectdb.train_transaction_pb
WHERE card4 IS NOT NULL AND card6 IS NOT NULL
GROUP BY card4, card6
ORDER BY fraud_rate DESC;

SELECT * FROM q3_results;

```

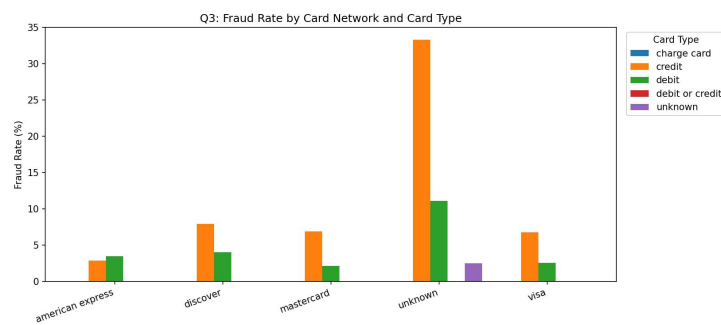


Figure 5: Fraud rate by card network and card type.

5.2.4 Daily Transaction Volume

Query example:

```

SELECT transaction_day,
COUNT(*) AS number_of_transactions
FROM fraud_transactions_hive
GROUP BY transaction_day
ORDER BY transaction_day;

```

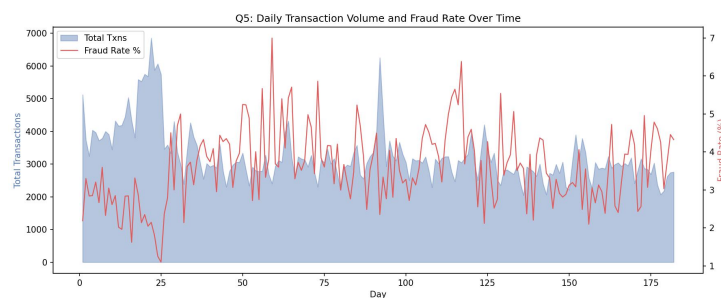


Figure 6: Daily transaction volume over time.

5.2.5 Email Domain Fraud Pattern

Query example:

```
SELECT email_domain,  
       ROUND(100 * AVG(CAST(is_fraud AS DOUBLE)), 2) AS fraud_rate  
FROM fraud_transactions_hive  
GROUP BY email_domain  
HAVING COUNT(*) > 50  
ORDER BY fraud_rate DESC  
LIMIT 10;
```

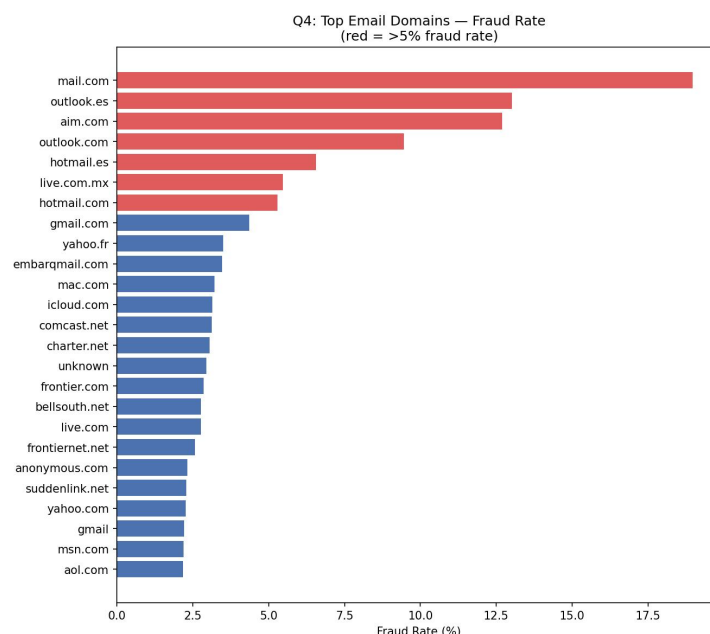


Figure 7: Email domain fraud pattern.

5.3 Interpretation

The charts reveal several important fraud patterns.

First, fraud risk varies strongly across product categories. Category C has the highest fraud rate. One possible explanation is that some product groups may be easier to monetize or harder to trace.

Second, the amount-band chart shows a non-linear pattern. Very small transactions under \$25 may be used as card-testing attempts, while larger transactions above \$250 may reflect attempts to extract more value before a card is blocked. The middle amount ranges are more stable.

Third, the combination of `card4` and `card6` is more informative than either variable alone. This means payment metadata should be used jointly.

Fourth, fraud activity changes over time. Daily monitoring is important because fraud pressure is not stable.

Finally, payer email domain also carries useful signal. It should not be used alone, but in combination with other transactional and identity features it improves the overall fraud-risk picture.

6 ML Modeling

We used Spark ML for predictive modeling. The task is binary classification: predict whether a transaction is fraudulent or not.

The two models presented are:

- **Gradient Boosted Trees (GBTClassifier);**
- **Random Forest (RandomForestClassifier).**

Both models are well suited for structured tabular fraud data because they can capture non-linear patterns and interactions between features.

6.1 Feature Extraction and Data Preprocessing

Before model training, the data was transformed into a machine-learning-ready format. The preprocessing pipeline included:

- joining transaction and identity data by transaction identifier;
- creating amount-based and temporal helper features;
- marking identity availability;
- filtering empty, constant, and unusable columns;
- handling missing numeric values through imputation;
- processing categorical variables depending on cardinality;
- encoding categorical variables for Spark ML;
- assembling all selected columns into one feature vector;
- applying feature reduction before training.

The main feature-selection summary was as follows:

Table 6: Feature inclusion and removal summary

Inclusion decision	Feature count
Included numeric	386
Included string	29
Included string top-k	10
Dropped high-null	9
Dropped high-cardinality low-coverage	5
Dropped constant or empty	3
Dropped dominant-value	1

This step was important because raw fraud data contains many noisy or sparse columns. Proper preprocessing improves model stability, reproducibility, and final quality.

6.2 Training and Fine-tuning

The models were trained with distributed Spark ML and hyperparameter tuning.

The balanced labeled dataset contained 40,987 rows. A reproducible 70/30 train-test split was used:

- 28,760 rows in the training split;
- 12,227 rows in the test split.

Cross-validation optimized area under precision-recall because the original fraud problem is highly imbalanced.

For **Gradient Boosted Trees**, the best parameters were:

- `maxDepth` = 7
- `stepSize` = 0.2

For **Random Forest**, the best parameters were:

- `maxDepth` = 8
- `numTrees` = 60

The full Stage III run took about 33 minutes. GBT cross-validation took about 20 minutes, while Random Forest tuning took about 6 minutes.

6.3 Evaluation

We compare GBT and RF models. GBT performed better than RF on all key reported metrics.

Table 7: Model evaluation results

Metric	GBT	RF
CV PR score	0.925607	0.890258
Test AUC-PR	0.931504	0.885948
Test AUC-ROC	0.922429	0.870194
Test F1	0.844644	0.794616
Test accuracy	0.844770	0.794962

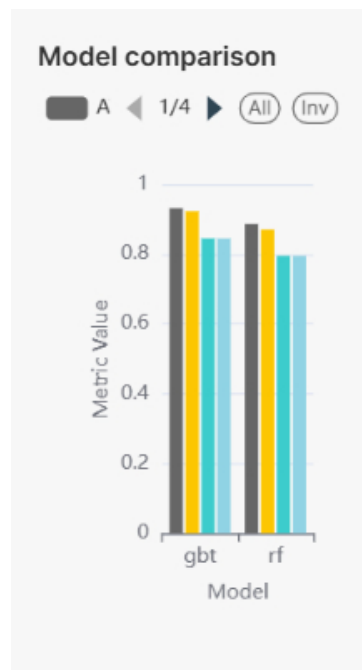


Figure 8: Model comparison for GBT and RF.

The comparison shows that **Gradient Boosted Trees** is the best model among the tested models.

We also did feature importance analysis. The strongest reported features were:

Table 8: Top reported feature importances

GBT		Random Forest	
Feature	Importance	Feature	Importance
v258_imputed	0.0524	v257_imputed	0.0432
log_transactionamt	0.0407	v258_imputed	0.0311
_imputed			
c14_imputed	0.0364	c14_imputed	0.0287
card2_imputed	0.0356	c13_imputed	0.0273

These results show that the machine learning stage not only produced strong prediction quality, but also helped identify the most influential fraud-related variables.

7 Data Presentation

The final results were presented in Apache Superset. The dashboard combines EDA insights and machine learning results in one interface that is suitable for final defence.

The dashboard is designed to answer three core questions:

- What are the main fraud patterns in the data?
- Which model performs better?
- Which features are important for fraud prediction?

7.1 The Description of the Dashboard

The dashboard is organized into three logical sections:

1. **Business framing:** project objective, pipeline stages, and the target proxy.
2. **Exploratory fraud-risk analysis:** product, amount, card, email-domain, and time-based charts.
3. **Predictive modeling:** model comparison, best-model block, feature importance, and feature-filtering summary.

The dashboard contains the following main blocks:

- fraud rate by product category;
- fraud rate and average amount by transaction amount band;
- fraud rate by card network and card type;
- email-domain fraud pattern;

- daily fraud activity over time;
- daily transaction volume over time;
- feature filtering summary;
- model comparison;
- best model highlight;
- GBT and RF feature importance.

7.2 Description of Each Chart

Table 9: Dashboard chart descriptions

Chart	Dataset	Purpose
Product category fraud rate	stage4_q1 product_risk	Shows which product categories have elevated fraud risk.
Transaction amount band risk	stage4_q2 amount_band_risk	Compares fraud rate by transaction amount range and highlights the non-linear pattern.
Card network/type risk	stage4_q3 card_risk	Compares fraud behavior by card network and card type combination.
Email-domain fraud pattern	stage4_q4 email_domain_risk	Highlights payer email domains with elevated fraud rate.
Daily fraud activity over time	stage4_q5 daily_risk	Shows how daily fraud pressure changes over time.
Daily transaction volume over time	stage4_q5 daily_risk	Shows trends in transaction activity by day.
Model comparison	stage4_model metrics_long	Compares GBT and Random Forest on key metrics.
Best model highlight	stage4_best model_summary	Displays the selected best model and its strongest metric.
GBT feature importance	stage4_feature importance_gbt	Explains the strongest GBT feature drivers.
RF feature importance	stage4_feature importance_rf	Explains the strongest Random Forest feature drivers.
Feature filtering summary	stage4_feature profile_summary	Summarizes included and removed feature groups before training.

7.3 Findings

- Fraud-risk behavior is concentrated in specific product categories, especially C.
- Transaction amount is non-linear: very small and larger transactions can both show elevated fraud risk.
- Payment metadata and card type carry useful predictive signal.
- Email domain is a meaningful risk indicator in raw EDA.
- Time variation exists and should be included in modeling.
- GBT achieved the best predictive quality, with test AUC-PR of 0.931504.
- Feature importance confirms that anonymized V/C features, transaction amount, and card-related attributes are important fraud drivers.

The main business conclusion is that fraud should not be evaluated using one variable only. A stronger fraud detection system should combine transaction amount, product category, card information, email domain, time-based signals, and ML-based risk scoring.

8 Conclusion

The project successfully demonstrates a reproducible end-to-end big data workflow for fraud-risk analytics. The pipeline starts from raw CSV files and moves through relational storage, HDFS ingestion, Hive and Spark SQL analysis, distributed Spark ML modeling, and final dashboard presentation.

The EDA stage revealed meaningful fraud-risk patterns in product categories, amount bands, email domains, payment metadata, and time. The machine learning stage showed that Gradient Boosted Trees outperformed Random Forest on the prepared fraud dataset. The dashboard brought these results together into a presentation-friendly format.

8.1 Limitations and Engineering Decisions

- **Limited business interpretation.** The exploratory analysis reveals fraud-risk patterns, but without real customer profiles, credit history, and repayment behavior, it is not possible to make strong conclusions about actual credit default.
- **Automatic schema inference has risk.** The schema was generated from CSV headers and sample values. This approach is convenient for a wide dataset with many columns, but sampling may miss rare values. To reduce this risk, a conservative fallback to TEXT was used when necessary.
- **Cluster resource constraints.** The feature filtering strategy and model preparation were shaped by the available distributed compute capacity, so some engineering choices were made with resource limits in mind.

8.2 Reflections on Own Work

This project showed that a real big data workflow depends on much more than model training. A large share of the work was connected with ingestion, schema handling, storage layout, validation, artifact checking, and presentation.

One of the most useful engineering decisions was to automate schema generation and preparation steps because the IEEE-CIS dataset contains hundreds of columns. Another important decision was to keep scripts rerunnable and stage outputs traceable, so that each part of the workflow could be checked independently.

From the documentation and reporting side, an important part of the work was organizing technical materials into one clear story.

8.3 Challenges and Difficulties

The main challenges were:

- local dataset files initially behaved like symlinks, so the actual CSV data had to be copied correctly to the cluster environment;
- cluster Python compatibility required script adjustments;
- the dataset contains many sparse and high-cardinality columns, so automated filtering was necessary;
- HiveServer2 and Beeline session issues made direct session initialization unstable in the cluster;
- relative HDFS paths initially caused incorrect or empty analytical results, so absolute paths had to be used;
- full model tuning had to be adapted to available cluster resources;
- dashboard charts had to remain readable while still showing enough business detail;
- all project artifacts had to be documented clearly for final defence.

8.4 Future work

- add model calibration and threshold analysis for practical fraud-review decisions;
- add more dashboard filters, such as product category, amount band, card type, and email domain;
- include a full prediction example with transaction ID, predicted label, and probability;
- preserve generated schemas, metrics, logs, and chart outputs under version control;

For business use, it is reasonable to combine the trained ML model with rule-based checks. For example, high amount, risky card metadata, suspicious email domain, and high predicted fraud probability can be combined into one unified fraud-risk score.

8.5 The Table of Contributions of Each Team Member

Project Task	Task Description	Ekaterina Petrova	Nikita Tsukanov	Olga Puzhalina	Alina Sukhoverkova	Deliverables	Avg. Hours
Data extraction	Collect IEEE-CIS dataset files and verify required CSVs	100%	0%	0%	0%	Raw CSV dataset, dataset notes	2
Database design and loading	Generate PostgreSQL schema, manage relational structure, and load source CSV data	100%	0%	0%	0%	Generated schema, PostgreSQL tables, loading scripts	5
HDFS import with Sqoop	Import PostgreSQL tables into HDFS as distributed analytical storage	100%	0%	0%	0%	Sqoop scripts, HDFS warehouse, schema files	4
Hive tables and analytical storage	Create external tables and prepare the partitioned and bucketed analytical layer	0%	0%	100%	0%	Hive DDL scripts, analytical tables	4
EDA queries	Run analytical queries, calculate fraud risk statistics, and export result outputs	0%	0%	100%	0%	EDA queries, CSV outputs, result tables	6
Feature preprocessing	Clean ML data, filter features, encode variables, and assemble feature vectors	0%	100%	0%	0%	Prepared ML dataset, feature filtering summary	5
ML modeling	Train distributed ML models, tune hyperparameters, evaluate results, and save outputs	0%	100%	0%	0%	GBT and RF models, metrics, feature importance	7
Prediction sample	Prepare and validate prediction results for selected transaction examples	0%	100%	0%	0%	Prediction output, probability and result table	2

Project Task	Task Description	Ekaterina Petrova	Nikita Tsukanov	Olga Puzhalina	Alina Sukhoverkova	Deliverables	Avg. Hours
Testing and validation	Check artifacts, screenshots, scripts, chart consistency, and evidence for the report	0%	0%	0%	100%	Validation notes, checked artifacts	4
Dashboard preparation	Build Superset charts, organize dashboard blocks, and prepare the final visual story	0%	0%	0%	100%	Superset dashboard, chart exports	5
Repository documentation	Document repository structure, usage steps, and stage outputs	0%	0%	0%	100%	README updates, documentation files	4
Final report	Organize the report, describe all stages, summarize findings, and prepare the final version	0%	0%	0%	100%	Final PDF report	6
Final presentation	Prepare presentation logic, dashboard demo, pipeline explanation, and speaking plan	25%	25%	25%	25%	Presentation slides, demo plan	4